

PYTHON FOR QUANT TRADERS · PART II

Math Tools — เครื่องมือคณิตศาสตร์สำหรับ Quant

บทที่ 11–17: Statistics · Time Series · OLS Regression · Cointegration · Optimization · Linear
Programming · Signals

จบ Part นี้ → สร้าง pair trading strategy ตั้งแต่ต้นจนได้ signal พร้อม trade

🚫 อ่าน Part นี้ยังไ้

🟢 **เพ็ญจบ Part I** — Part นี้คือจุดที่ได้เริ่มยากขึ้นชัดเจน · กลยุทธ์ที่ได้ผล: อ่าน **กล่อง** 💡 **แก่นของบทนี้** ให้เข้าใจก่อน แล้วค่อยลงโค้ด · เจอฟังก์ชันสถิติที่ไม่รู้จัก (`adfuller` , `coint` , `minimize`) **ไม่ต้องเข้าใจคณิตศาสตร์ข้างในทั้งหมด** — เข้าใจว่า "ใส่อะไรเข้าไป ได้อะไรออกมา แล้วตีความยังไง" ก็พอสำหรับรอบแรก

🟢 **มีพื้น stat/econometrics** — เนื้อทฤษฎีคุ้นอยู่แล้ว เข้าได้ · ที่ควรอ่านคือ **กล่อง** ⚠️ **กับดัก** ทั้งหมด โดยเฉพาะ **บทที่ 14 เรื่อง null hypothesis กลับหัว** (p เล็ก = ชั่วดี) ที่คนอ่านผิดกันบ่อยมากจนเลือกคู่ที่ไม่ cointegrate มาเทรดทั้งพอร์ต · และ **บทที่ 13 เรื่องข้อมูลล้นมาจาก pandas ไม่ใช่ statsmodels**

📌 สิ่งที่จะทำได้เมื่อจบ PART II

- หาคู่หุ้นที่ cointegrate กัน ด้วย ADF test และ Johansen test
 - ประมาณค่า (estimate) hedge ratio β ด้วย OLS regression (statsmodels)
 - สร้าง spread และคำนวณ z-score สำหรับ entry/exit signal
 - Maximize Sharpe ratio ด้วย `scipy.optimize`
 - กำหนด portfolio weight constraints ด้วย Linear Programming
- ทุกอย่างนี้คือ **core engine** ของ Statistical Arbitrage

🏃 Running Example ตลอด Part II — Pair Trading: BTC Perp Bybit ↔ BTC Perp Binance

เราจะสร้าง pair trading strategy ที่ละขั้นตอนตลอด 7 บท:

บท 11 → ตรวจสอบ distribution ของ spread · บท 12 → ทดสอบ stationarity · บท 13 → ประมาณ β ด้วย OLS

บท 14 → ยืนยัน cointegration · บท 15 → optimize position size · บท 16 → ใส่ capital constraint · บท 17 → สร้าง signal pipeline

บทที่ 11: Statistics & Probability เชิงลึก

แก่นของบทนี้

สถิติใน Part 0 เป็นแนวคิด — บทนี้แปลงเป็น Python จริง ด้วย `scipy.stats` และ `numpy` เน้น 3 เรื่องที่ Quant ใช้จริง: hypothesis testing (กลยุทธ์นี้ดีจริงหรือแค่โชค?), VaR/CVaR (ความเสี่ยงสูงสุดคือเท่าไร?), และ fat tail (ทำไมต้องระวัง Black Swan?)

11.1 Hypothesis Testing — กลยุทธ์ดีจริงหรือแค่โชค?

สมมติฐาน: H_0 (null) = "กลยุทธ์ไม่ดีกว่าสุ่ม" vs H_1 = "กลยุทธ์ดีกว่าสุ่มจริงๆ"

```
from scipy import stats
import numpy as np

# ผลตอบแทนรายวัน 252 วัน จากกลยุทธ์
np.random.seed(42)
strategy_returns = np.random.normal(loc=0.0008, scale=0.015, size=252)

# t-test: H0 = mean return = 0 (ไม่ดีกว่าสุ่ม)
t_stat, p_value = stats.ttest_1samp(strategy_returns, popmean=0)

print(f"Mean Return: {strategy_returns.mean():.4f} ({strategy_returns.mean()*252:.1%}/year)")
print(f"t-statistic: {t_stat:.3f}")
print(f"p-value: {p_value:.4f}")

if p_value < 0.05:
    print("✓ มีนัยสำคัญ — กลยุทธ์น่าจะดีกว่าสุ่ม (p < 5%)")
else:
    print("✗ ไม่มีนัยสำคัญ — อาจเป็นแค่โชค")
```

► OUTPUT

```
Mean Return: 0.0007 (18.7%/year)
t-statistic: 0.814
p-value: 0.4167
✗ ไม่มีนัยสำคัญ — อาจเป็นแค่โชค
```

ทำไม Return ดูดีแต่ p-value ยังสูง?

20% ต่อปีดูน่าสนใจ แต่ถ้า σ สูง (1.5%/วัน) — noise มากกว่า signal มาก
ต้องมีข้อมูลมากพอ (หลาย 100 วัน) ถึงจะ "แยก" signal ออกจาก noise ได้

$$t = \frac{\bar{r}}{\sigma/\sqrt{n}} \rightarrow \text{ยิ่ง } n \text{ มาก } t \text{ ยิ่งใหญ่} \rightarrow p\text{-value} \text{ ยิ่งเล็ก}$$

⚠️ แล้วถ้าผ่านล่ะ — $p < 0.05$ แปลว่า "ทำเงินได้" หรือเปล่า

ไม่ · และนี่คือความเข้าใจผิดที่แพงที่สุดในบรรดาความเข้าใจผิดทั้งหมดในหนังสือเล่มนี้

สังเกตบรรทัดที่เขียนว่า `popmean=0` ให้ดี — เรากำลังถามว่า "ผลตอบแทนต่างจากศูนย์ไหม" · แต่คำถามที่เราสนใจจริงๆ คือ "ผลตอบแทนชนะค่าธรรมเนียมไหม" · สองคำถามนี้ไม่เหมือนกัน และสถิติตอบให้แค่คำถามที่เราถาม

สูตร $t = \bar{r} / (\sigma/\sqrt{n})$ ในกล่องข้างบนบอกความจริงอีกด้านที่ต้องเห็นให้ครบ: ตัวหารเล็กลงเรื่อย ๆ ตาม $\sqrt{n}$ · แปลว่าถ้าเก็บข้อมูลมากพอ edge เล็กแค่ไหนก็ผลักให้ $p < 0.05$ ได้ทั้งนั้น — รวมถึง edge ที่เล็กกว่าค่าธรรมเนียมหลายเท่า · $p\text{-value}$ ไม่เคยรู้ว่าคุณจ่ายค่าธรรมเนียมเท่าไร เพราะไม่มีใครบอกมัน

📖 หัวข้อ 25.4b ใน Part IV แสดงตัวอย่างที่วัดเป็นตัวเลขได้: กลยุทธ์ที่ $t = +6.62$ · $p < 0.0001$ (มี edge จริงแน่นอน) แต่ขาดทุนปีละ 27,000 บาท เพราะ edge เล็กกว่าต้นทุน · พร้อมวิธีแก้ที่ใช้เวลาแก้แค่บรรทัดเดียว

11.2 VaR และ CVaR — ความเสี่ยงสูงสุดคือเท่าไร?

$$\text{VaR}_{95\%} = \text{percentile}_{5\%}(r) \quad \text{CVaR}_{95\%} = E[r \mid r \leq \text{VaR}_{95\%}]$$

```
# VaR = ขาดทุนสูงสุดใน 95% ของวันทั้งหมด
# CVaR = ขาดทุนเฉลี่ยในวันที่แย่ที่สุด 5%
```

```
returns = strategy_returns # จากข้างบน
capital = 1_000_000        # ทุน 1 ล้านบาท
```

```
var_95 = np.percentile(returns, 5)
cvar_95 = returns[returns <= var_95].mean()
```

```
print(f"VaR 95%: {var_95:.2%} → ขาดทุนไม่เกิน {abs(var_95)*capital:,.0f} บาท/วัน (95% ของเวลา)
print(f"CVaR 95%: {cvar_95:.2%} → เฉลี่ยขาดทุน {abs(cvar_95)*capital:,.0f} บาท ในวันที่แย่ที่สุด 5%
```

หมายเหตุ CVaR: ต้องการ sample เพียงพอ

CVaR 95% ใช้เฉพาะ return ที่อยู่ใน worst 5% — ถ้ามีข้อมูล 100 วัน จะใช้แค่ 5 วัน

ถ้าข้อมูลน้อยกว่า 60 วัน CVaR อาจไม่น่าเชื่อถือ ควรใช้ `if len(tail) >= 5: cvar = tail.mean()`

หรือเพิ่ม `n_tail = max(1, int(len(returns) * 0.05))` เพื่อรับประกัน minimum sample

11.3 Fat Tail — ทำไม Normal Distribution ถึงอันตราย?

```

from scipy import stats

# เปรียบ Normal กับ t-distribution (fat tail)
normal_99 = stats.norm.ppf(0.01) # -2.33σ
t5_99      = stats.t.ppf(0.01, df=5) # -3.36σ (df=5 = fat tail)

print(f"Normal 1% VaR: {normal_99:.2f}σ")
print(f"t(df=5) 1% VaR: {t5_99:.2f}σ")
print(f"ใช้ Normal จะ underestimate ความเสี่ยง {t5_99/normal_99:.1f}x")

```

อย่าเชื่อ Normal Distribution สำหรับ crypto และช่วง crisis

BTC crash 50%+ เกิดหลายครั้ง — ถ้าใช้ Normal Distribution จะบอกว่า "แทบเป็นไปไม่ได้" (เกิน 10σ — เหตุการณ์ที่สูตร Normal บอกว่าแทบเป็นไปไม่ได้เลยในช่วงชีวิตเรา)

ในความจริง: ราคาสินทรัพย์มี "fat tail" = เหตุการณ์สุดขั้วเกิดบ่อยกว่า Normal คาด

ใช้ t-distribution, Cornish-Fisher, หรือ historical simulation แทน Normal สำหรับ risk management

ตัวอย่าง: สรุป Return Statistics แบบมืออาชีพ

```
def return_stats(returns, name="Strategy", capital=1_000_000):
    """สรุป statistics สำหรับ return series"""
    r = np.array(returns)
    ann = 252

    mean_daily = r.mean()                # return เฉลี่ยต่อวัน
    std_daily = r.std()                   # ความผันผวนต่อวัน
    sharpe = mean_daily / std_daily * np.sqrt(ann)  # Sharpe รายปี (คูณ  $\sqrt{252}$ )
    skew = stats.skew(r)                  # ความเบ้ (ลบ = หางซ้ายยาว = อันตราย)
    kurt = stats.kurtosis(r)              # excess kurtosis (บวก = fat tail)
    var95 = np.percentile(r, 5)           # จุดตัด worst 5% ของวัน
    cvar95 = r[r <= var95].mean()         # ค่าเฉลี่ยของวันที่แย่กว่า VaR

    equity = capital * np.cumprod(1 + r)   # เส้นเงินทุนสะสมรายวัน
    rolling_max = np.maximum.accumulate(equity)  # จุดสูงสุดที่เคยไปถึง ณ แต่ละวัน
    max_dd = ((equity - rolling_max) / rolling_max).min()  # % ร่วงจากยอดที่ลึกที่สุด

    print(f"=== {name} ===")
    print(f"Annual Return: {mean_daily*ann:.1%}   Sharpe: {sharpe:.2f}")
    print(f"Annual Vol: {std_daily*np.sqrt(ann):.1%}   Max DD: {max_dd:.1%}")
    print(f"Skewness: {skew:.2f}   Kurtosis: {kurt:.2f}   (Normal: 0, 0)")
    print(f"VaR 95%: {var95:.2%}   CVaR 95%: {cvar95:.2%}")

return_stats(strategy_returns)
```

► แบบฝึกหัด: เปรียบ Sharpe ของสองกลยุทธ์

บทที่ 12: Time Series Basics

แก่นของบทนี้

ข้อมูลราคาเรียงตามเวลา มีคุณสมบัติพิเศษที่ข้อมูลทั่วไปไม่มี: แต่ละจุดสัมพันธ์กับจุดก่อนหน้า (autocorrelation) และอาจ "ล่องลอย" ไปเรื่อยๆ โดยไม่กลับ (non-stationary) การเข้าใจ 2 เรื่องนี้คือ กุญแจของ Statistical Arbitrage ทั้งหมด

12.1 Stationarity — ล่องลอยหรือวนกลับ?

Stationary series: mean และ variance คงที่ตลอดเวลา — "วนกลับหาค่ากลาง"

Non-stationary series: mean เปลี่ยนตามเวลา — "ล่องลอยไม่มีทิศทาง" (random walk)

```
import numpy as np

# Non-stationary: Random Walk (ราคา BTC)
np.random.seed(0)
random_walk = np.cumsum(np.random.normal(0, 1, 300))

# Stationary: Spread ระหว่างสองตลาด
spread = np.random.normal(0, 1, 300) # OU-like (mean-reverting)

# สังเกต: random_walk drift ออกไปเรื่อยๆ
#         spread วนอยู่รอบ 0
print(f"Random Walk range: {random_walk.min():.1f} to {random_walk.max():.1f}")
print(f"Spread range:      {spread.min():.1f} to {spread.max():.1f}")
```

ทำไม Stationarity ถึงสำคัญสำหรับ Pair Trading?

ถ้า spread ไม่stationary แปลว่าไม่มีแรงดึงกลับ — z-score ที่สูงอยู่อาจสูงต่อไปเรื่อยๆ โดยไม่กลับมา เท่ากับเปิด trade แล้วอาจขาดทุนไม่หยุด

ถ้า spread stationary → มีแรงดึงกลับ → z-score สูงแล้ว "ต้องลง" ในที่สุด → เป็น basis ของ arb

เราต้องพิสูจน์ก่อนเสมอว่า spread เป็น stationary จริง — ไม่ใช่แค่ดูกราฟ

12.2 ADF Test — ทดสอบ Stationarity

Augmented Dickey-Fuller (ADF) test: H_0 = non-stationary (random walk) vs H_1 = stationary

```

from statsmodels.tsa.stattools import adfuller

def test_stationarity(series, name="Series"):
    """รัน ADF test และแสดงผลแบบเข้าใจง่าย"""
    result = adfuller(series, autolag="AIC")
    adf_stat = result[0]
    p_value = result[1]
    crit_vals = result[4]

    print(f"\n--- ADF Test: {name} ---")
    print(f"ADF Statistic: {adf_stat:.4f}")
    print(f"p-value: {p_value:.4f}")
    print(f"Critical (1%): {crit_vals['1%']:.4f}")
    print(f"Critical (5%): {crit_vals['5%']:.4f}")

    if p_value < 0.05:
        print(f"✓ STATIONARY (p={p_value:.3f} < 0.05) – กลับหาค่ากลาง")
    else:
        print(f"✗ NON-STATIONARY (p={p_value:.3f} > 0.05) – random walk")
    return p_value < 0.05

# ทดสอบกับข้อมูลจาก pair
test_stationarity(random_walk, "BTC Price (Random Walk)")
test_stationarity(spread, "BTC Spread (Mean-Reverting)")

```

12.3 Autocorrelation — ราคาวันนี้สัมพันธ์กับเมื่อวานแค่ไหน?

```

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# ACF/PACF ของ returns (stationary)
plot_acf(spread, lags=30, ax=axes[0][0], title="ACF – Spread (Stationary)")
plot_pacf(spread, lags=30, ax=axes[0][1], title="PACF – Spread")

# ACF/PACF ของ raw price (non-stationary)
plot_acf(random_walk, lags=30, ax=axes[1][0], title="ACF – Price (Non-stationary)")
plot_pacf(random_walk, lags=30, ax=axes[1][1], title="PACF – Price")

plt.tight_layout()
plt.savefig("acf_comparison.png", dpi=150)
plt.show()

```


อ่าน ACF plot อย่างไร?

Pattern ที่เห็น	แปลว่า	ทำอะไรต่อ
ค่าลด exponentially ใน ACF	AR process (mean-reverting) ✓	ใช้เป็น spread ได้
ค่าสูงทุก lag ไม่ลด	Non-stationary (random walk)	ต้อง difference ก่อน
ค่าตัดที่ lag 1 ใน ACF	MA(1) process	ต้องใส่ MA term ใน model

► แบบฝึกหัด: ทดสอบ stationarity ของ spread จริง

บทที่ 13: OLS Regression

แก่นของบทนี้

OLS (Ordinary Least Squares) เป็นเครื่องมือหลักสำหรับประมาณ hedge ratio β — อัตราส่วนที่บอกว่าต้อง long/short ในสัดส่วนเท่าไรเพื่อสร้าง spread ที่ stationary — คำว่า 'ประมาณ' ในที่นี้หมายถึงการคำนวณหาค่าที่เหมาะสมที่สุดจากข้อมูล ไม่ใช้การเดาคร่าว ๆ สูตร matrix form:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

13.1 OLS ด้วย statsmodels

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# จำลองราคา BTC สองตลาด — เหรียญเดียวกัน คนละตลาด
np.random.seed(42)
n = 252
btc_bybit = 65000 + np.cumsum(np.random.normal(0, 100, n))

# ส่วนต่างราคาสองตลาด (spread) ไม่ได้สุ่มลอย ๆ — มันดึงกลับเข้าค่ากลาง
# ใช้ OU process ตัวเดียวกับบทที่ 12: กำไรของ pair trade มาจากแรงดึงนี้
kappa, sigma_ou = 0.15, 25.0          # kappa สูง = ดึงกลับเร็ว
spread_ou = np.zeros(n)
for t in range(1, n):
    spread_ou[t] = (spread_ou[t-1]
                    + kappa * (0 - spread_ou[t-1])
                    + np.random.normal(0, sigma_ou))

# Binance = ราคา Bybit + basis 0.03% + spread ที่ดึงกลับได้
btc_binance = btc_bybit * 1.0003 + spread_ou

# OLS: log(P_binance) = alpha + beta * log(P_bybit) + epsilon
log_bybit = np.log(btc_bybit)
log_binance = np.log(btc_binance)

# ใส่ชื่อคอลัมน์ด้วย DataFrame — เพื่อเรียก params ด้วย "ชื่อ" ได้ทีหลัง
X = sm.add_constant(pd.DataFrame({"log_bybit": log_bybit}))
y = log_binance
```

```

model = sm.OLS(y, X).fit()

alpha = model.params['const']
beta = model.params['log_bybit']
resid = model.resid          # epsilon (spread)
r2 = model.rsquared
p_beta = model.pvalues['log_bybit']

print(f"alpha (intercept): {alpha:.6f}")
print(f"beta (hedge ratio): {beta:.6f}")
print(f"R²: {r2:.4f}")
print(f"p-value (beta): {p_beta:.6f}")
print(f"\nSpread (residuals) stats:")
print(f"  Mean: {resid.mean():.6f}")
print(f"  Std: {resid.std():.6f}")

```

► OUTPUT

```

alpha (intercept): 0.207854
beta (hedge ratio): 0.981260
R²: 0.9903
p-value (beta): 0.000000

Spread (residuals) stats:
  Mean: 0.000000
  Std: 0.000695

```

🔴**อ่านว่า:** บรรทัดที่ต้องสังเกต: `sm.add_constant(pd.DataFrame({"log_bybit": log_bybit}))` — ทำไมต้องห่อด้วย DataFrame? เพราะถ้าส่ง **numpy array** เข้าไปตรง ๆ ผลลัพธ์ `model.params` จะเป็น array ที่ไม่มีชื่อเรียกได้แค่ตำแหน่ง (`params[0]` , `params[1]`) → บรรทัด `params['const']` จะพังทันที (`IndexError`) .พอห่อเป็น DataFrame ที่มีชื่อคอลัมน์ statsmodels จะคืน **Series** ที่มี label ให้ → เรียกด้วยชื่อได้ อ่านโค้ดรู้เรื่องกว่า และไม่พลาดสลับตำแหน่ง α กับ β

⚠ **กับดักที่เจอบ่อย — "ชื่อ" มาจาก pandas ไม่ใช่ statsmodels**

นี่คือความสับสน numpy ↔ pandas ที่คลาสสิกที่สุดในงาน quant: **statsmodels** ไม่ได้ตั้งชื่อคอลัมน์ให้คุณ — มันแค่ส่งต่อชื่อที่มาจาก pandas ถ้าต้นทางไม่มีชื่อ ปลายทางก็ไม่มี

วิธีเช็คเร็ว ๆ ก่อนเรียกด้วยชื่อ: `print(type(model.params))` — ถ้าได้ `ndarray` ให้ใช้ตำแหน่ง ถ้าได้ `Series` ใช้ชื่อได้

13.2 ดีความผล OLS

Output	ความหมาย	ค่าที่ดี
β	Hedge ratio — Long 1 บาท Bybit ต้อง Short β บาท Binance	ใกล้ 1.0 สำหรับ same asset
α	Constant spread ระหว่างสองตลาด	ขึ้นอยู่กับ pair
R^2	สัดส่วน variance ที่ X อธิบายได้	> 0.85 สำหรับ good pair
p-value (β)	มั่นใจแค่ไหนว่า $\beta \neq 0$	< 0.001 สำหรับ cointegrated pair
Residuals	Spread = ส่วนที่ไม่สัมพันธ์กับ X — นี่คือนี่ที่เราจะ trade	Mean ≈ 0 , Stationary

13.3 Rolling OLS — β ที่ Adaptive

β ไม่คงที่ตลอดเวลา ต้องอัปเดตสม่ำเสมอ

```
from statsmodels.regression.rolling import RollingOLS

# Rolling OLS window 60 วัน
window = 60
# X ตัวเดิมจาก 13.1 — เป็น DataFrame ที่มีชื่อคอลัมน์อยู่แล้ว
X = sm.add_constant(pd.DataFrame({"log_bybit": log_bybit}))

rolling_model = RollingOLS(log_binance, X, window=window).fit()
rolling_beta = rolling_model.params['log_bybit']

# สร้าง rolling spread
rolling_alpha = rolling_model.params['const']
spread = log_binance - (rolling_alpha + rolling_beta * log_bybit)

print(f"Beta range: {rolling_beta.dropna().min():.4f} to {rolling_beta.dropna().max():.4f}")
print(f"Spread Std: {spread.std():.6f}")
```

► OUTPUT

```
Beta range: 0.9071 to 1.1820
Spread Std: 0.000718
```

📌 **อ่านว่า:** อ่าน `rolling_model.params['log_bybit']` ว่า: "หีบค่า β ของทุกวัน ออกมา" — ต่างจาก 13.1 ที่ได้ β ตัวเดียว ตรงนี้ได้เป็น **Series ยาวเท่าจำนวนวัน** (60 วันแรกเป็น NaN เพราะยังนับไม่ครบหน้าต่าง) · ดูช่วง 0.907–1.182 = β ขยับได้ถึง $\pm 18\%$ รอบ 1.0 นี่แหละความหมายของ " **β ไม่คงที่**" ที่หัวข้อนี้จะบอก — ถ้า hedge ด้วย β ตัวเดียว ค้างไว้ตลอดปี คุณจะ hedge ผิดสัดส่วนเป็นระยะ ๆ

window เลือกอย่างไร?

- สั้นเกินไป (เช่น 10 วัน): β volatile มาก, noise สูง $\rightarrow$ signal ผิดพลาดบ่อย

- ยาวเกินไป (เช่น 500 วัน): β ตอบสนองช้ามาก → พลาด regime change
- ดี: window $\approx 3-5$ เท่าของ half-life ของ spread — เรียนวิธีหา half-life ในบทที่ 14

ตัวอย่าง: สร้าง Z-score จาก OLS Residuals

```
def build_spread(price_a, price_b, window=60):
    """
    สร้าง spread และ z-score จาก OLS rolling regression
    ทำ 3 ขั้นตอน: (1) หา beta → (2) สร้าง spread → (3) คำนวณ z-score
    Returns: DataFrame ของ spread, beta, zscore
    """
    log_a = np.log(price_a)          # แปลงเป็น log price ทั้งคู่
    log_b = np.log(price_b)

    # — ขั้น 1: หา hedge ratio (beta) ด้วย rolling OLS —
    X = sm.add_constant(log_b)       # เพิ่มคอลัมน์ค่าคงที่ (intercept) ให้ regression
    roll = RollingOLS(log_a, X, window=window).fit()
    beta = roll.params.iloc[:, 1]     # คอลัมน์ 0 คือ const (จาก add_constant), คอลัมน์ 1 คือ beta
    alpha = roll.params['const']      # intercept

    # — ขั้น 2: spread = ส่วนที่ "เหลือ" หลังหักความสัมพันธ์ออก —
    predicted = alpha + beta * log_b  # ราคา A ที่ "ควรจะเป็น" ตามราคา B
    spread = log_a - predicted        # ของจริง - ที่ควรเป็น = ความเพี้ยน

    # — ขั้น 3: z-score ของ spread (สูตรเดิมจาก Part I) —
    roll_mean = spread.rolling(window).mean()
    roll_std = spread.rolling(window).std()
    zscore = (spread - roll_mean) / roll_std

    return pd.DataFrame({
        "spread": spread,
        "beta": beta,
        "zscore": zscore
    })

df_pair = build_spread(
    pd.Series(btc_bybit),
    pd.Series(btc_binance),
    window=60
)
print(df_pair.tail())
print(f"\nCurrent Z-score: {df_pair['zscore'].iloc[-1]:.2f}")
```

► OUTPUT

	spread	beta	zscore
247	-0.000126	0.933411	-0.497962
248	0.000514	0.933770	0.348072
249	0.000118	0.932216	-0.160151
250	-0.000725	0.929067	-1.226366

251 -0.000047 0.929990 -0.320480

Current Z-score: -0.32

🔗**อ่านว่า:** แกนของฟังก์ชันนี้อยู่ที่สองบรรทัด: `predicted = alpha + beta * log_b` แล้ว `spread = log_a - predicted` — อ่านว่า "ราคา A ที่ควรจะเป็นตามราคา B" แล้วเอา "ของจริง ลบ ที่ควรเป็น". ส่วนที่เหลือคือความเพี้ยน และความเพี้ยนนี้แหละคือสิ่งที่เราเทรด ไม่ใช่ตัวราคา

⚠️ สังเกตสองบรรทัดที่หยาบค่าคนละแบบ — ไม่ใช่ความมั่งง่าย

ในฟังก์ชันเดียวกันมีทั้ง `roll.params['const']` (ใช้ชื่อ) และ `roll.params.iloc[:, 1]` (ใช้ตำแหน่ง) — เพราะ `sm.add_constant()` ตั้งชื่อให้เฉพาะคอลัมน์ที่มันเติมเข้าไปเองคือ `const`. ส่วนคอลัมน์ข้อมูลเดิมจะได้ชื่อก็ต่อเมื่อ Series ที่ส่งเข้ามามีชื่ออยู่แล้ว — ที่นี้ส่ง `pd.Series(btc_binance)` ซึ่งไม่มีชื่อ จึงต้องอ้างด้วยตำแหน่ง

🔗**อยากให้เรียกด้วยชื่อได้ทั้งคู่** ให้ตั้งชื่อ Series ตั้งแต่ต้น: `pd.Series(btc_binance, name="log_b")` — หลักการเดียวกับที่เจอในบทที่ 13

► แบบฝึกหัด: interpret OLS output

บทที่ 14: Cointegration & Z-score Signal

แก่นของบทนี้

Correlation บอกว่าสองสิ่งเดินไปทิศเดียวกัน — Cointegration บอกว่าสองสิ่ง *ผูกกันระยะยาว* แม้ระยะสั้นจะแยกออกจากกันได้ Cointegration = ข้อพิสูจน์ว่า spread มีแรงดึงกลับจริง = basis ของ Statistical Arbitrage ทั้งหมด

14.1 Correlation vs Cointegration

	Correlation สูง	Cointegrated
หมายถึง	เดินทิศเดียวกันในระยะสั้น	ผูกกันในระยะยาว มี "แรงดึง"
Spread	อาจ drift ออกไปเรื่อยๆ	กลับหาค่ากลางเสมอ
ตัวอย่าง	หุ้นสอง sector เดียวกัน	BTC สองตลาด, ADR vs local
Trade ได้ไหม?	เสี่ยง — spread อาจไม่กลับ	ใช่ — spread ต้องกลับ (ในทฤษฎี)

14.2 Engle-Granger Test — 2 ขั้นตอน

วิธี Engle-Granger: (1) OLS หา residuals → (2) ADF test บน residuals

```
from statsmodels.tsa.stattools import adfuller, coint

# วิธีที่ 1: Engle-Granger ด้วย coint() โดยตรง
score, pvalue, crit_vals = coint(np.log(btc_bybit), np.log(btc_binance))

print(f"Cointegration test:")
print(f"  Test statistic: {score:.4f}")
print(f"  p-value:         {pvalue:.4f}")
print(f"  Critical (5%):   {crit_vals[1]:.4f}")

if pvalue < 0.05:
    print(f"✓ COINTEGRATED — spread มีแรงดึงกลับจริง")
else:
    print(f"✗ NOT COINTEGRATED — อย่า trade pair นี้!")

# วิธีที่ 2: ADF บน residuals (เข้าใจกระบวนการมากขึ้น)
log_a = np.log(btc_bybit)
```



```
log_b = np.log(btc_binance)
X = sm.add_constant(log_b)
residuals = sm.OLS(log_a, X).fit().resid
adf_resid = adfuller(residuals)
print(f"\nADF on residuals p-value: {adf_resid[1]:.4f}")
```

► OUTPUT

Cointegration test:

Test statistic: -4.7762

p-value: 0.0004

Critical (5%): -3.3606

✓ COINTEGRATED — spread มีแรงดึงกลับจริง

ADF on residuals p-value: 0.0001

📌 **อ่านว่า:** p-value = 0.0004 แล้วสรุปว่า "cointegrated" — ทิศทางนี้กลับหัวจากที่คนส่วนใหญ่คุ้น · เพราะสมมติฐานตั้งต้น (null) ของ `coint()` คือ "คู่นี้ *ไม่* cointegrate" · p-value เล็ก = หลักฐานแรงพอจะล้มสมมติฐานนั้น = คู่นี้ cointegrate จริง · จำสั้น ๆ: **p เล็ก = ขาวดี**

❌ **ข้อผิดพลาดที่พบบ่อยที่สุดในเรื่องนี้ — อ่าน p-value กลับด้าน**

ถ้าแปลอ่านแบบ " $p > 0.05 = \text{ผ่าน}$ " เหมือนการทดสอบทั่วไป คุณจะเลือกคู่นี้ *ไม่* cointegrate มาเทรดทั้งหมด และทิ้งคู่นี้ทิ้งไปหมด — ผลคือพอร์ตที่ spread ไม่มีแรงดึงกลับ ซึ่งคือการเดาทิศทางสั้น ๆ โดยที่คุณคิดว่ากำลังทำ market-neutral อยู่

สาเหตุที่พลาด: การทดสอบตระกูลนี้ (ADF, coint, KPSS บางตัว) ตั้ง null เป็น "สิ่งที่ *ไม่* ยากได้" — ต่างจาก t-test ที่ null คือ "ไม่มีผล" · **เปิดคู่มือดู null ทุกครั้งก่อนตีความ** อย่าเดาจากความเคยชิน

14.3 Johansen Test — สำหรับ 3+ Pairs

```
from statsmodels.tsa.vector_ar.vecm import coint_johansen

# Johansen ทดสอบ cointegration ระหว่าง 3 series พร้อมกัน
btc_okx = 65000 + np.cumsum(np.random.normal(0, 100, n)) # ตลาดที่ 3

data = np.column_stack([np.log(btc_bybit), np.log(btc_binance), np.log(btc_okx)])
result = coint_johansen(data, det_order=0, k_ar_diff=1)

print("Johansen Trace Statistics:")
for i in range(3):
    trace_stat = result.lr1[i]
    crit_5pct = result.cvt[i, 1]
    print(f" r≤{i}: trace={trace_stat:.2f}, crit(5%)={crit_5pct:.2f} "
          f"{'✓' if trace_stat > crit_5pct else 'x'}")
```

```
# r=0: ทดสอบว่ามีอย่างน้อย 1 cointegrating vector
# r≤1: ทดสอบว่ามีอย่างน้อย 2
```

► OUTPUT

Johansen Trace Statistics:

```
r≤0: trace=35.43, crit(5%)=29.80 ✓
r≤1: trace=10.34, crit(5%)=15.49 ✗
r≤2: trace=0.79, crit(5%)=3.84 ✗
```

 อ่านผล Johansen — มันคือ "บันได" ไม่ใช่คำตอบเดียว

```
r≤0: trace=35.43, crit=29.80 ✓ ← ล้มได้
r≤1: trace=10.34, crit=15.49 ✗ ← ล้มไม่ได้ → หยุดตรงนี้
r≤2: trace= 0.79, crit= 3.84 ✗
```

วิธีอ่าน: ไล่จากบนลงล่าง หยุดที่แถวแรกที่ล้มไม่ได้ แล้วเลขแถวนั้นคือคำตอบ

1. $r \leq 0$ ถามว่า "มี cointegrating vector **0 อัน** ใช่ไหม" · trace 35.43 > 29.80 → **ล้มข้อนี้ได้** แปลว่ามีอย่างน้อย 1 อัน → ไต่ขึ้นแถวถัดไป
2. $r \leq 1$ ถามว่า "มีไม่เกิน **1 อัน** ใช่ไหม" · trace 10.34 < 15.49 → **ล้มไม่ได้** → ยอมรับว่าจริง
3. สรุป: มี cointegrating vector **1 อันพอดี** — คือ 3 ตลาดนี้มีความสัมพันธ์ระยะยาวชุดเดียว (ตามที่ควรเป็น เพราะเราสร้าง Bybit กับ Binance ให้เกาะกัน ส่วน OKX เป็นเส้นอิสระ)

 ✓ ในผลลัพธ์แปลว่า "ล้ม null ได้" ไม่ได้แปลว่า "ดี" — ✓ ที่แถวสุดท้ายทุกแถวจะแปลว่าไม่มีความสัมพันธ์ระยะยาวเลย

🔵 [รอบสอง] ทำไมต้อง Johansen ทั้งที่มี Engle-Granger แล้ว

Engle-Granger ต้องเลือกก่อนว่าใครเป็นตัวแปรตาม (`coint(a, b)` กับ `coint(b, a)` ให้ p-value ไม่เท่ากัน!) — พอมี 3 สินทรัพย์ขึ้นไป การเลือกนี้กลายเป็นการตัดสินใจตามอำเภอใจที่กระทบผลลัพธ์ · Johansen ทดสอบทุกทิศพร้อมกันโดยไม่ต้องเลือก และบอกได้ด้วยว่ามีความสัมพันธ์กี่ชุดไม่ใช่แค่ "มี/ไม่มี"

ในทางปฏิบัติ: 2 ขา → Engle-Granger พอ (อ่านง่ายกว่า) · 3 ขาขึ้นไป (เช่น basket ข้ามตลาด, curve trade) → Johansen เท่านั้น · และ `k_ar_diff` ที่ตั้งเป็น 1 นั้นควรเลือกด้วย information criterion จริง ๆ ไม่ใช่ค่าคงที่ — ตั้งผิดทำให้ผลพลิกได้

14.4 Half-Life — Spread กลับเร็วแค่ไหน?

Half-life คือเวลาที่ spread ใช้กลับมากครึ่งทางจากค่าสุดขีด — ใช้เลือก window และ holding period

$$\text{Half-life} = \frac{-\ln(2)}{\ln(\hat{\phi})} \approx \frac{-\ln(2)}{\hat{\phi} - 1}$$

ซ้าย = สูตรแม่นยำ (โค้ดด้านล่างใช้สูตรนี้) · ขวา = ค่าประมาณ (ใช้ได้เมื่อ $\hat{\phi}$ ใกล้ 1)

```
def calc_half-life(spread):
    """คำนวณ half-life ของ mean-reverting spread"""
    spread_lag = pd.Series(spread).shift(1)
    delta      = pd.Series(spread) - spread_lag
    df_hl = pd.DataFrame({"delta": delta, "lag": spread_lag}).dropna()

    X = sm.add_constant(df_hl["lag"])
    phi = sm.OLS(df_hl["delta"], X).fit().params["lag"]

    if phi >= 0:
        return np.inf # ไม่ mean-reverting (random walk หรือ explosive)
    if phi <= -1:
        return np.inf # Oscillating — ไม่ใช่ mean-reverting จริง
    half-life = -np.log(2) / np.log(1 + phi) # สูตรแม่นยำ: phi จาก regression = φ-1 ดังนั้น 1
    return half-life

hl = calc_half-life(resid)
print(f"Half-life: {hl:.1f} วัน")
print(f"แนะนำ: rolling window ≈ {hl*3:.0f} วัน, hold ไม่เกิน {hl*2:.0f} วัน")
```

► OUTPUT

Half-life: 3.8 วัน

แนะนำ: rolling window ≈ 11 วัน, hold ไม่เกิน 8 วัน

🔴**อ่านว่า:** อ่านผลนี้ว่า: "spread ใช้เวลา **3.8 วัน** เพื่อเดินทางกลับมารั้งทางจากจุดที่เหวี่ยงออกไป" — ตัวเลขนี้ตั้งค่าให้คุณสองอย่างทันที: **window** ต้องยาวพอเห็นรอบการดึงกลับ (≈3 เท่าของ half-life) และ **holding period** ต้องไม่ยาวเกินจนถือค้างหลังกำไรวิ่งมาแล้ว (≈2 เท่า) · ⚠️ ถ้าได้ half-life สั้นมาก (<1 วัน) แปลว่า spread เป็น noise ไม่ใช่ mean-reversion จริง — ค่าคำแนะนำจะกลายเป็น 0 วันซึ่งเทรดไม่ได้ นั่นคือสัญญาณว่า pair นี้ไม่ควรเทรด ไม่ใช่ว่าโค้ดพัง

ตัวอย่าง: Signal Generation Pipeline ครบวงจร

```
def pair_signal(price_a, price_b, window=60,
               entry_z=2.0, exit_z=0.5):
    """
    สร้าง trading signal สำหรับ pair trading
    Returns: DataFrame พร้อม signal column
    """
    df = build_spread(price_a, price_b, window)
    z = df["zscore"]

    signals = pd.Series("HOLD", index=z.index)
    signals[z > entry_z] = "SHORT_A_LONG_B" # spread แพงเกินไป
    signals[z < -entry_z] = "LONG_A_SHORT_B" # spread ถูกเกินไป
    signals[abs(z) < exit_z] = "EXIT"

    df["signal"] = signals
    # ⚠ สำคัญมาก: signal ที่ได้จาก df.index[t] ใช้ข้อมูลถึงวัน t
    # ต้อง .shift(1) ใน backtest ก่อน multiply ด้วย return เสมอ
    return df

result = pair_signal(
    pd.Series(btc_bybit),
    pd.Series(btc_binance),
    window=60, entry_z=2.0, exit_z=0.5
)
print(result[["spread", "beta", "zscore", "signal"]].tail(10))
```

บทที่ 15: Optimization

แก่นของบทนี้

Optimization คือการหาค่าตัวแปร (เช่น portfolio weights, position size) ที่ทำให้ objective function (เช่น Sharpe ratio) ดีที่สุด ภายใต้ constraints (เช่น weights รวม = 1) Python ทำได้ด้วย `scipy.optimize.minimize`

$$\max_{\mathbf{w}} \text{Sharpe}(\mathbf{w}) = \frac{\mathbf{w}^\top \boldsymbol{\mu} - r_f}{\sqrt{\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}}} \quad \text{s.t.} \quad \sum w_i = 1, w_i \geq 0$$

15.1 scipy.optimize.minimize — API

```
from scipy.optimize import minimize
import numpy as np

# สมมติว่าต้องการหา x ที่ทำให้ f(x) = (x-3)^2 น้อยที่สุด
def f(x):
    return (x[0] - 3)**2

result = minimize(f, x0=[0], method="SLSQP")
print(f"Optimal x: {result.x[0]:.4f} (คาดหวัง: 3.0)")
print(f"Min f(x): {result.fun:.6f}")
print(f"Success: {result.success}")
```

► OUTPUT

```
Optimal x: 3.0000 (คาดหวัง: 3.0)
Min f(x): 0.000000
Success: True
```

📖 **อ่านว่า:** อ่าน `minimize(f, x0=[0])` ว่า: "หาค่า x ที่ทำให้ f น้อยที่สุด โดยเริ่มเดาที่ x=0" — สิ่งที่ต้องเข้าใจคือ optimizer **ไม่รู้จักรัสตรของคุณ** มันแค่ลองค่า ดูว่าผลออกมาเท่าไร แล้วขยับไปทางที่ผลลดลง ทำซ้ำจนขยับแล้วไม่ดีขึ้น. `x0` จึงสำคัญ — จุดเริ่มต่างกันอาจได้คำตอบต่างกันถ้าฟังก์ชันมีหลุมหลายหลุม

⚠ **result.success** ต้องเช็คทุกครั้ง — optimizer โกหกแบบเงิบ ๆ ได้

`minimize()` คืนค่ากลับมาเสมอไม่ว่าจะหาเจอหรือไม่·ถ้ามันไม่converge ค่าใน `result.x` จะเป็นตำแหน่งที่มันยอมแพ้ไม่ใช่คำตอบ — และถ้าคุณไม่เช็ค `result.success` คุณจะเอา "น้ำหนักพอร์ตที่ไม่ใช่คำตอบ" ไปลงเงินจริง โดยไม่มีerrorใดๆเตือน

15.2 Sharpe Ratio Maximization

```
def sharpe_maximize(returns_df, risk_free=0.0, periods=252):
    """
    หา portfolio weights ที่ maximize Sharpe ratio
    returns_df: DataFrame ของ daily returns (cols = assets)
    """
    n = len(returns_df.columns)
    mu = returns_df.mean().values * periods          # annualized mean
    sigma = returns_df.cov().values * periods         # annualized cov

    def neg_sharpe(weights):
        """คืนค่า negative Sharpe (minimize = maximize Sharpe)"""
        port_ret = weights @ mu
        port_var = weights @ sigma @ weights
        return -(port_ret - risk_free) / np.sqrt(port_var)

    # Constraints
    constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1}] # sum = 1
    bounds = [(0, 1)] * n # no short
    x0 = np.ones(n) / n # equally weighted

    result = minimize(neg_sharpe, x0,
                      method="SLSQP",
                      bounds=bounds,
                      constraints=constraints)

    if not result.success:
        print("Warning: optimizer did not converge")

    weights = result.x
    port_ret = weights @ mu
    port_vol = np.sqrt(weights @ sigma @ weights)

    print("=== Max Sharpe Portfolio ===")
    for name, w in zip(returns_df.columns, weights):
        print(f" {name}: {w:.1%}")
    print(f" Expected Return: {port_ret:.1%}/yr")
    print(f" Volatility: {port_vol:.1%}/yr")
    print(f" Sharpe Ratio: {(port_ret-risk_free)/port_vol:.2f}")
    return weights
```

```
# ทดสอบ
np.random.seed(0)
rets = pd.DataFrame(np.random.multivariate_normal(
    [0.001, 0.0007, 0.0012],
    [[0.0004, 0.0002, 0.0001],
     [0.0002, 0.0003, 0.0001],
     [0.0001, 0.0001, 0.0005]], 252),
    columns=["BTC", "ETH", "SOL"])

optimal_w = sharpe_maximize(rets)
```

🧑‍🎓 ฝึกอ่าน 2 ท่อนที่ดูสลับที่สุดในโค้ดข้างบน

```
# ท่อนที่ 1
port_ret = weights @ mu
# ท่อนที่ 2
constraints = [{"type": "eq", "fun": lambda w: np.sum(w) - 1}]
```

ท่อนที่ 1: เครื่องหมาย @ คือ "คูณแบบเวกเตอร์/เมทริกซ์" — `weights @ mu` อ่านว่า "เอาน้ำหนักแต่ละตัว คูณ return คาดหวังของ asset นั้น แล้วบวกรวมกัน" เท่ากับเขียน loop เองว่า:

```
port_ret = 0
for w, m in zip(weights, mu):
    port_ret = port_ret + w * m
```

ท่อนที่ 2: อ่านจากในออกนอก:

1. `lambda w: np.sum(w) - 1` — ฟังก์ชันจิ๋ว: "รับ w มา แล้วคืนค่า ผลรวมของ w ลบ 1"
2. `"type": "eq"` — บอก optimizer ว่าค่าที่ฟังก์ชันนี้คืน **ต้องเท่ากับ 0** → แปลว่า $\text{sum}(w) - 1 = 0 \rightarrow \text{sum}(w) = 1$ = "ลงทุนเต็มพอดิไม่ขาดไม่เกิน"
3. `[ { ... } ]` — เป็น list ของ dict เพราะใส่ได้หลายเงื่อนไข (scipy กำหนดรูปแบบนี้)

นี่คือ "ภาษาเฉพาะ" ของ scipy — เจอครั้งแรกทุกคนงง พออ่านออกแล้วจะเห็นว่ามันแค่บอกว่า *"เงื่อนไข: น้ำหนักรวมกัน ต้องเป็น 1"*

15.3 Kelly Criterion — Optimal Position Size

$$f^* = \frac{\mu - r_f}{\sigma^2} \quad (\text{continuous Kelly})$$

```
def kelly_fraction(mean_ret, std_ret, risk_free=0.0, fraction=0.25):
    """
    คำนวณ Kelly fraction สำหรับ continuous distribution
    fraction: ใช้ 0.25 (Quarter-Kelly) เพื่อความปลอดภัย
```

```

"""
kelly_full = (mean_ret - risk_free) / (std_ret ** 2)
kelly_frac = kelly_full * fraction

print(f"Full Kelly:      {kelly_full:.2f}x")
print(f"Quarter Kelly: {kelly_frac:.2f}x ← แนะนำ")
return kelly_frac

# ตัวอย่าง: กลยุทธ์ที่มี Sharpe = 1.5
daily_mean = 0.001      # 0.1% ต่อวัน
daily_std  = 0.015      # 1.5% ต่อวัน
f = kelly_fraction(daily_mean, daily_std)

```

Kelly ที่ใช้ที่นี้ = Continuous Kelly (สำหรับ return กระจาย Normal)

สูตร $f^* = \frac{\mu - r_f}{\sigma^2}$ เหมาะกับ asset ที่ return กระจายแบบ Normal distribution ต่อเนื่อง
 สำหรับ **discrete bet** เช่น Binary outcome (win ได้ b เท่า, lose 1): $f^* = \frac{pb - (1-p)}{b}$

ใน Quant Trading ส่วนใหญ่ใช้ continuous Kelly หรือบ่อยกว่านั้นใช้ **Half Kelly ($f^*/2$)** หรือ **Quarter Kelly ($f^*/4$)** เพราะ:

- Return จริงไม่ Normal (fat tail)
- Estimate μ และ σ มี error เสมอ
- Full Kelly มี drawdown รุนแรงมากในช่วง streak ของ losing trades

อย่าใช้ Full Kelly ในทางปฏิบัติ

Full Kelly maximize geometric growth ในทฤษฎี แต่ในทางปฏิบัติ:

- ประมาณ parameters จากข้อมูลมีความผิดพลาด → Full Kelly อาจ oversize มาก
- Drawdown จาก Full Kelly ใหญ่มาก (50%+ เป็นปกติ) → ทนไม่ไหว

ใช้ **Quarter-Kelly ($f^*/4$)** เป็นจุดเริ่มต้น — Part IV จะพาไปลึกขึ้นในเรื่องนี้

ตัวอย่าง: Mean-Variance Optimization (Efficient Frontier)

```
def efficient_frontier(returns_df, n_points=100):
    """
    สร้าง efficient frontier:
    สำหรับ return เป้าหมายแต่ละระดับ → หาพอร์ตที่ vol ต่ำสุด
    """
    n = len(returns_df.columns)
    mu = returns_df.mean().values * 252 # expected return รายปี
    sigma = returns_df.cov().values * 252 # covariance รายปี

    # ไล่ return เป้าหมาย จากต่ำสุดถึงสูงสุด เป็น n_points จุด
    target_returns = np.linspace(mu.min(), mu.max(), n_points)

    def portfolio_variance(w):
        """สิ่งที่ต้องการ minimize: ความแปรปรวนของพอร์ต"""
        return w @ sigma @ w

    frontier_vols = [] # เก็บ vol ต่ำสุดของแต่ละเป้า
    for target in target_returns:
        # เงื่อนไข 1: น้ำหนักรวม = 1 (ลงทุนเต็มพอดี)
        sum_to_one = {"type": "eq", "fun": lambda w: np.sum(w) - 1}

        # เงื่อนไข 2: return ของพอร์ต = target ของรอบนี้
        # (t=target จำเป็น! ดูล่องกับดักด้านล่าง)
        hit_target = {"type": "eq", "fun": lambda w, t=target: w @ mu - t}

        result = minimize(portfolio_variance,
                          x0=np.ones(n) / n, # เริ่มจากน้ำหนักเท่ากันทุกตัว
                          method="SLSQP",
                          bounds=[(0, 1)] * n, # ห้าม short (0 ≤ w ≤ 1)
                          constraints=[sum_to_one, hit_target])

        if result.success:
            min_vol = np.sqrt(result.fun) # ได้ variance ต่ำสุด → ถอดรากเป็น vol
        else:
            min_vol = np.nan # แก้มิได้ (เป้าสูง/ต่ำเกิน) → เว้นจุดนี้
        frontier_vols.append(min_vol)

    return target_returns, np.array(frontier_vols)

rets_arr, vols_arr = efficient_frontier(rets)

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(vols_arr, rets_arr, color="#0d9488", linewidth=2, label="Efficient Frontier")
ax.set_xlabel("Annual Volatility")
```

```
ax.set_ylabel("Annual Return")
ax.set_title("Efficient Frontier (BTC, ETH, SOL)")
ax.legend()
plt.tight_layout()
plt.show()
```

⚠ กับดัก lambda ใน loop — ทำไมต้องเขียน `lambda w, t=target`

ถ้าเขียน `lambda w: w @ mu - target` เฉย ๆ — lambda จะ "จำชื่อตัวแปร" ไม่ใช่ "จำค่า" พอ loop จบ `target` ชี้ไปที่ค่าสุดท้ายแล้ว ทุก lambda ที่สร้างไว้จะใช้ค่าเดียวกันหมด → frontier ผิดทั้งเส้นแบบเงียบ ๆ ไม่มี error

ทำ `t=target` คือการ "ถ่ายสำเนาค่า ณ รอบนั้น" เก็บไว้ใน `t` ของ lambda ตัวนั้นเอง — นี่เป็นกับดัก Python คลาสสิกที่แม้แต่ AI ยังเขียนพลาดเป็นครั้งคราว เจอ `lambda` ใน `loop` เมื่อไหร่ ให้มองหา = แบบนี้เสมอ

► แบบฝึกหัด: Minimize Portfolio Variance (Min-Vol Portfolio)

บทที่ 16: Linear Programming

แก่นของบทนี้

Linear Programming (LP) คือ optimization ที่ทั้ง objective function และ constraints เป็น linear — เร็วกว่า nonlinear optimization มาก ใช้ใน Quant สำหรับ: กำหนด portfolio weight ภายใต้หลาย constraint พร้อมกัน, minimize transaction cost, sizing หลาย position พร้อมกัน

$$\min_{\mathbf{w}} \mathbf{c}^\top \mathbf{w} \quad \text{s.t.} \quad \mathbf{A}\mathbf{w} \leq \mathbf{b}, \mathbf{A}_{eq}\mathbf{w} = \mathbf{b}_{eq}, \mathbf{lb} \leq \mathbf{w} \leq \mathbf{ub}$$

16.1 scipy.linprog — API

```
from scipy.optimize import linprog

# ตัวอย่างง่าย: minimize cost ของการซื้อ 3 assets
# minimize: 2*w1 + 3*w2 + 1.5*w3
# subject to: w1+w2+w3 = 1 (fully invested)
#             w1 >= 0.1 (min 10% BTC)
#             w2 >= 0.1 (min 10% ETH)

c = [2.0, 3.0, 1.5]                # objective: cost coefficients
A_eq = [[1, 1, 1]]                 # equality: sum = 1
b_eq = [1]
bounds = [(0.1, 0.6), (0.1, 0.5), (0, 0.8)] # min/max ของแต่ละ w

result = linprog(c, A_eq=A_eq, b_eq=b_eq,
                 bounds=bounds, method="highs")

print(f"Optimal weights: {result.x.round(3)}")
print(f"Minimum cost: {result.fun:.4f}")
print(f"Success: {result.success}")
```

16.2 Portfolio LP — ใส่ Constraints หลายอย่างพร้อมกัน

```
from scipy.optimize import linprog
import numpy as np

def lp_portfolio(expected_returns, min_return=0.10,
                 max_weights=None, min_weights=None,
```

```

        max_sector=None, sectors=None):
    """
    Minimize portfolio variance (approximated as linear)
    ด้วย LP – เหมาะสำหรับ constraint-heavy problem

    expected_returns: array ของ expected annual return แต่ละ asset
    min_return: minimum portfolio return ที่ต้องการ
    """
    n = len(expected_returns)

    # Objective: minimize negative return (= maximize return)
    # Linear approximation – ใช้สำหรับ constraint-heavy problems
    c = -np.array(expected_returns)

    # Equality: sum(w) = 1
    A_eq = [np.ones(n)]
    b_eq = [1.0]

    # Inequality: min return constraint
    # w @ returns >= min_return → -w @ returns <= -min_return
    A_ub = [-np.array(expected_returns)]
    b_ub = [-min_return]

    # Bounds: default 0-1 (long only)
    if min_weights is None: min_weights = [0.0] * n
    if max_weights is None: max_weights = [1.0] * n
    bounds = list(zip(min_weights, max_weights))

    result = linprog(c, A_ub=A_ub, b_ub=b_ub,
                     A_eq=A_eq, b_eq=b_eq,
                     bounds=bounds, method="highs")

    return result

# ตัวอย่าง: 5 assets
assets = ["BTC", "ETH", "SOL", "AAPL", "GOLD"]
returns = [0.35, 0.25, 0.40, 0.12, 0.08]

result = lp_portfolio(
    returns,
    min_return = 0.20,
    min_weights = [0.05]*5,      # min 5% ทุก asset
    max_weights = [0.40]*5,      # max 40% ทุก asset
)

print("LP Portfolio Allocation:")
for asset, w in zip(assets, result.x):

```

```
print(f" {asset:5s}: {w:.1%}")
print(f"Expected Return: {np.dot(result.x, returns):.1%}")
```

16.3 PuLP — สำหรับ Integer Programming

```
# pip install pulp
from pulp import *

# Integer LP: เลือก k หุ้นที่ดีที่สุดจาก n ตัว
scores = {"BTC": 0.85, "ETH": 0.70, "SOL": 0.60,
          "AAPL": 0.55, "GOLD": 0.40}
costs = {"BTC": 5000, "ETH": 2000, "SOL": 500,
         "AAPL": 1000, "GOLD": 800}
budget = 8000 # งบ 8000 บาท

prob = LpProblem("SelectAssets", LpMaximize)

# ตัวแปร binary: 1=เลือก, 0=ไม่เลือก
x = {a: LpVariable(f"x_{a}", cat="Binary") for a in scores}

# Objective: maximize total score
prob += lpSum(scores[a] * x[a] for a in scores)

# Constraints
prob += lpSum(costs[a] * x[a] for a in scores) <= budget # budget
prob += lpSum(x[a] for a in scores) <= 3 # เลือกได้ max 3 ตัว

prob.solve(PULP_CBC_CMD(msg=0))

print("Selected assets:")
for a in scores:
    if x[a].value() == 1:
        print(f" {a}: score={scores[a]}, cost={costs[a]}")
```

ตัวอย่าง: Position Sizing LP สำหรับ Pair Trading

```
# กำหนด: มี 3 pairs พร้อม trade, capital จำกัด
# objective: maximize expected PnL
# constraints: total capital, max drawdown per pair, correlation limit

pairs      = ["BTC-Bybit/Binance", "ETH-OKX/Bybit", "SOL-Binance/OKX"]
exp_pnl    = [0.15, 0.12, 0.20] # expected annual return
max_dd     = [0.05, 0.08, 0.10] # max expected drawdown
capital    = 1_000_000           # total capital บาท

# Minimize negative PnL = maximize PnL
c          = [-e for e in exp_pnl] # negate for minimization

# sum(allocation) = 1.0 (fully allocated)
A_eq = [[1, 1, 1]]
b_eq = [1.0]

# เงื่อนไข: drawdown ที่คาดหวังของแต่ละ pair ≤ 5% ของ capital
# allocation_i × max_dd_i ≤ 0.05 — แยกกันทีละ pair = matrix แนวทาง
A_ub = np.diag(max_dd) # [[0.05, 0, 0],
                        # [0, 0.08, 0],
                        # [0, 0, 0.10]]

b_ub = [0.05] * 3

bounds = [(0.0, 0.5)] * 3 # max 50% ต่อ pair

result = linprog(c, A_ub=A_ub, b_ub=b_ub,
                 A_eq=A_eq, b_eq=b_eq,
                 bounds=bounds, method="highs")

print("Optimal Position Sizing:")
for pair, alloc in zip(pairs, result.x):
    print(f" {pair}: {alloc:.1%} = ฿{alloc*capital:,.0f}")
print(f"Expected Portfolio Return: {-result.fun: .1%}")
```

AI มักสร้าง matrix แนวทางด้วย nested comprehension — ฟังก์ชัน

```
A_ub = [[max_dd[i] if j==i else 0 for j in range(3)] for i in range(3)]
```

comprehension ซ้อนสองชั้น — อ่านจาก **for** ขวาสุดก่อน: `for i` สร้างทีละแถว แล้วค่อยอ่าน `for j` ที่อยู่ถัดมาทางซ้าย เพื่อสร้างทีละช่องในแถวนั้น:

1. `for i in range(3)` (ชั้นนอก, ขวาสุด) — สร้าง 3 แถว แถวละหนึ่ง `i`
2. `for j in range(3)` (ชั้นใน) — แต่ละแถวมี 3 ช่อง ช่องละหนึ่ง `j`
3. `max_dd[i] if j==i else 0` — แต่ละช่อง: "เอาค่า `max_dd` ถ้าอยู่บนเส้นทแยง (แถว=คอลัมน์) **ไม่**งั้นใส่ 0"

ผลคือ matrix เดียวกับ `np.diag(max_dd)` เป๊ะ — เวอร์ชัน `np.diag` สั้นกว่า อ่านง่ายกว่า และผิดยากกว่า ถ้า AI เขียนแบบ nested มา ขอให้มันแก้เป็น `np.diag` ได้เลย

► แบบฝึกหัด: LP Constraint สำหรับ Sector Limits

บทที่ 17: Signals & Filters

แก่นของบทนี้

Signal คือ "เมื่อไหร่ควร trade" — บทนี้รวม 3 เครื่องมือ: **Bollinger Bands** (signal จาก volatility), **Kalman Filter** (track β แบบ adaptive), และ **Signal Pipeline** (รวมหลาย signal เข้าด้วยกัน) ซึ่งเป็น boilerplate ที่ใช้ได้กับทุก strategy

17.1 Bollinger Bands — Signal จาก Volatility

```
def bollinger_bands(series, window=20, n_std=2):
    """คำนวณ Bollinger Bands"""
    s = pd.Series(series)
    mean = s.rolling(window).mean()
    std = s.rolling(window).std()
    upper = mean + n_std * std
    lower = mean - n_std * std
    return pd.DataFrame({"mid": mean, "upper": upper, "lower": lower,
                        "pct_b": (s - lower) / (upper - lower)})

# pct_b: 0 = ที่ lower band, 1 = ที่ upper band
# signal: SHORT เมื่อ pct_b > 1, LONG เมื่อ pct_b < 0
bb = bollinger_bands(spread, window=20, n_std=2)
signal = pd.Series("HOLD", index=bb.index)
signal[bb["pct_b"] > 1.0] = "SHORT"
signal[bb["pct_b"] < 0.0] = "LONG"
```

17.2 Kalman Filter — Track β แบบ Adaptive

Kalman Filter ประมาณ β แบบ real-time โดยอัปเดตทุกครั้งที่มีข้อมูลใหม่ — ดีกว่า rolling OLS ตรงที่ไม่มี window cutoff และ smooth กว่า

Kalman Filter ทำงานสองจังหวะ — "เดา แล้วแก้เดา" ซ้ำไปเรื่อย ๆ

ลองนึกถึงการเดาน้ำหนักเพื่อนทุกครั้งที่เราเจอ:

จังหวะ 1 — เดา (Predict): "วันนี้น่าจะหนักเท่าเดิมจากครั้งก่อน แต่เวลาผ่านไปแล้ว ความมั่นใจของเราลดลง นิดหน่อย"

จังหวะ 2 — แก้เดา (Update): เห็นข้อมูลใหม่ (เพื่อนตัวจริง) → เทียบกับที่เดา → ถ้าต่างกัน ขยับค่าเดาเข้าหาความจริง ตามสัดส่วนความมั่นใจ — มั่นใจค่าเดาเดิมมากก็ขยับนิดเดียว ไม่มั่นใจก็ขยับเยอะ

ตัวเลข "สัดส่วนการขยับ" นี้คือ **Kalman Gain (K)** — หัวใจของทั้ง algorithm มีแค่นี้

```
def kalman_hedge_ratio(price_a, price_b,
                       delta=1e-4, vt=1e-3):
    """
    Track hedge ratio beta ด้วย Kalman Filter
    delta: process noise (ใหญ่ = ยอมให้ beta เปลี่ยนเร็ว)
    vt: observation noise (ใหญ่ = ราคา มี noise มาก เชื่อข้อมูลใหม่น้อยลง)
    Returns: array ของ beta ณ แต่ละเวลา
    """
    n = len(price_a)
    beta = np.zeros(n) # เตรียมที่เก็บ beta ของทุกวัน
    P = np.ones(n) # P = "ความไม่มั่นใจ" ในค่า beta (variance ของค่าเดา)
    beta[0] = 1.0 # วันแรก: เดาไว้ก่อนว่า beta = 1

    for t in range(1, n):
        # — จังหวะ 1: เดา (Predict) —
        # beta วันนี้น่าจะเท่าเมื่อวาน แต่ความไม่มั่นใจเพิ่มขึ้นตาม delta
        P_pred = P[t-1] + delta

        # — จังหวะ 2: แก้เดา (Update) ด้วยข้อมูลใหม่ —
        x = price_b[t] # ราคาตลาด B วันนี้ (ตัวพยากรณ์)

        y_pred = beta[t-1] * x # ราคา A "ที่ควรเป็น" ตามค่าเดาเดิม
        innov = price_a[t] - y_pred # ของจริง - ที่เดา = "เดาพลาดเท่าไร"

        S = x**2 * P_pred + vt # ความแปรปรวนรวมของความพลาด
                                # (จากความไม่มั่นใจใน beta + noise ของราคา)
        K = P_pred * x / S # Kalman Gain: "ควรขยับค่าเดากี่ %"
                                # ไม่มั่นใจมาก (P สูง) → K ใหญ่ → ขยับเยอะ
                                # ราคา noise มาก (vt สูง) → K เล็ก → ขยับนิดเดียว

        beta[t] = beta[t-1] + K * innov # ขยับ beta เข้าหาความจริง ตามสัดส่วน K
        P[t] = (1 - K * x) * P_pred # แก้เดาแล้ว → ความมั่นใจเพิ่มขึ้น (P ลดลง)

    return beta

# ทดสอบ
log_a = np.log(btc_bybit)
log_b = np.log(btc_binance)
kf_beta = kalman_hedge_ratio(log_a, log_b)

print(f"Kalman Beta (latest 5): {kf_beta[-5:].round(6)}")
```

17.3 Signal Pipeline — รวมทุกอย่างเข้าด้วยกัน

```
class PairTradingSignal:
    """
    Signal pipeline สำหรับ Pair Trading
    รวม: OLS hedge ratio, z-score, Bollinger filter
    """
    def __init__(self, window=60, entry_z=2.0, exit_z=0.5):
        self.window = window
        self.entry_z = entry_z
        self.exit_z = exit_z

    def fit(self, price_a: pd.Series, price_b: pd.Series):
        """ประมาณ parameters จากข้อมูลประวัติ"""
        self.price_a = price_a
        self.price_b = price_b

        # OLS spread
        self.df = build_spread(price_a, price_b, self.window)

        # Half-life
        self.halflife = calc_halflife(self.df["spread"].dropna())
        print(f"Half-life: {self.halflife:.1f} วัน")

        # ADF test
        adf_p = adfuller(self.df["spread"].dropna())[1]
        self.is_stationary = adf_p < 0.05
        print(f"Stationary: {self.is_stationary} (p={adf_p:.3f})")
        return self

    def signal(self) -> pd.Series:
        """สร้าง trading signal"""
        z = self.df["zscore"]
        sig = pd.Series("HOLD", index=z.index)
        sig[z > self.entry_z] = "SHORT_A"
        sig[z < -self.entry_z] = "LONG_A"
        sig[abs(z) < self.exit_z] = "EXIT"
        return sig

    def summary(self):
        """แสดง summary ของ current state"""
        current_z = self.df["zscore"].iloc[-1]
        current_b = self.df["beta"].iloc[-1]
        sig = self.signal().iloc[-1]
        print(f"=== Pair Signal Summary ===")
        print(f"Current Z-score: {current_z:.3f}")
        print(f"Current Beta: {current_b:.6f}")
```

```

print(f"Signal:          {sig}")
print(f"Is Stationary:    {self.is_stationary}")

# ใช้งาน
pipeline = PairTradingSignal(window=60, entry_z=2.0, exit_z=0.5)
pipeline.fit(pd.Series(btc_bybit), pd.Series(btc_binance))
pipeline.summary()

```

จบ Part II — สิ่งที่ได้แล้ว

- ทดสอบ stationarity ด้วย ADF test ✓
- ยืนยัน cointegration ด้วย Engle-Granger และ Johansen ✓
- ประเมิน β ด้วย OLS และ Kalman Filter ✓
- สร้าง z-score signal สำหรับ entry/exit ✓
- Maximize Sharpe ratio ด้วย `scipy.optimize` ✓
- กำหนด portfolio constraint ด้วย Linear Programming ✓
- รวมทุกอย่างเป็น `PairTradingSignal` class ✓

ใน **Part III** เราจะ refactor code นี้ให้เป็น OOP ที่ maintainable และ extensible

⚠ คำเตือน: ตัวเลขใน Part II คือ In-Sample — อย่าเชื่อ 100%

ยินดีด้วย — มาถึงตรงนี้แสดงว่าคุณสร้าง strategy แรกได้แล้ว แต่มีเรื่องสำคัญหนึ่งอย่างที่ต้องรู้ก่อนใช้เงินจริง: Sharpe 2.0+ และ Return 15%+ ที่เห็นใน Part II ล้วน **คำนวณจากข้อมูลชุดเดียวกับที่ optimize parameter** (in-sample)

ในชีวิตจริง (out-of-sample) ผลลัพธ์มักต่ำกว่า **50–80%** เพราะ:

- ตลาดเปลี่ยน — pattern ในอดีตไม่ซ้ำแบบในอนาคต
- Parameter ที่ดีในอดีต = overfit กับ noise ในอดีต
- Transaction cost ที่ลืมใส่ กินไป 20–50% ของ gross PnL

Part IV จะสอน Walk-Forward Validation — วิธีเดียวที่ยืนยันได้ว่า strategy มี edge จริง ไม่ใช่แค่ overfit อย่าใช้เงินจริงจนกว่าจะผ่านการทดสอบใน Part IV

